

# NPFL103: Information Retrieval (2)

Positional index, Dictionary data structures, Tolerant retrieval

Pavel Pecina

`pecina@ufal.mff.cuni.cz`

Lecturer

Institute of Formal and Applied Linguistics  
Faculty of Mathematics and Physics  
Charles University

Based on slides by Hinrich Schütze, University of Stuttgart.

# Contents

## Positional index

- Phrase queries

- Proximity search

## Dictionary data structures

- Arrays

- Hashes and trees

## Tolerant retrieval

- Wildcard queries

- Permuterm index

- $k$ -gram index

Positional index

## Phrase queries

- ▶ We answer a query such as [STANFORD UNIVERSITY] – as a phrase.
- ▶ “*The inventor Stanford Ovshinsky never went to university*” → not a match
- ▶ The concept of phrase query has proven easily understood by users.
- ▶ About 10% of web queries are phrase queries.
- ▶ Consequence for inverted index:  
It no longer suffices to store docIDs in postings lists.
- ▶ Solution: extending the inverted index with positional information: positional index

## Positional indexes

- ▶ Postings lists in a **nonpositional** index: each posting is just a docID
- ▶ Postings lists in a **positional** index: each posting is a docID and **a list of positions**

## Positional indexes: Example

Query: “ $to_1$   $be_2$   $or_3$   $not_4$   $to_5$   $be_6$ ”

TO, 993427:

1:  $\langle 7, 18, 33, 72, 86, 231 \rangle$ ;

2:  $\langle 1, 17, 74, 222, 255 \rangle$ ;

4:  $\langle 8, 16, 190, 429, 433 \rangle$ ;

5:  $\langle 363, 367 \rangle$ ;

7:  $\langle 13, 23, 191 \rangle$ ; ...

BE, 178239:

1:  $\langle 17, 25 \rangle$ ;

4:  $\langle 17, 191, 291, 430, 434 \rangle$ ;

5:  $\langle 14, 19, 101 \rangle$ ; ...

Document 4 is a match!

## Proximity search

- ▶ We just saw how to use a positional index for phrase searches.
- ▶ We can also use it for proximity search.
- ▶ **For example:** *employment /4 place*
  - ▶  $\Rightarrow$  find all documents that contain EMPLOYMENT and PLACE within 4 words of each other.
- ▶ “Employment agencies that place healthcare workers are seeing growth”  $\rightarrow$  **a hit.**
- ▶ “Employment agencies that have learned to adapt now place healthcare workers”  $\rightarrow$  **not a hit.**

## Proximity search

- ▶ Use the positional index
- ▶ Simplest algorithm: look at all combinations of positions of (i) EMPLOYMENT in document and (ii) PLACE in document
- ▶ Very inefficient for frequent words, especially stop words
- ▶ Note that we want to return the actual matching positions, not just a list of documents.
- ▶ This is important for dynamic summaries etc.



## “Proximity” intersection

POSITIONALINTERSECT( $p_1, p_2, k$ )

```

1  answer  $\leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $l \leftarrow \langle \rangle$ 
5           $pp_1 \leftarrow \text{positions}(p_1)$ 
6           $pp_2 \leftarrow \text{positions}(p_2)$ 
7          while  $pp_1 \neq \text{NIL}$ 
8              do while  $pp_2 \neq \text{NIL}$ 
9                  do if  $|\text{pos}(pp_1) - \text{pos}(pp_2)| \leq k$ 
10                     then  $\text{ADD}(l, \text{pos}(pp_2))$ 
11                     else if  $\text{pos}(pp_2) > \text{pos}(pp_1)$ 
12                         then break
13                      $pp_2 \leftarrow \text{next}(pp_2)$ 
14                     while  $l \neq \langle \rangle$  and  $|l[0] - \text{pos}(pp_1)| > k$ 
15                         do  $\text{DELETE}(l[0])$ 
16                     for each  $ps \in l$ 
17                         do  $\text{ADD}(\text{answer}, \langle \text{docID}(p_1), \text{pos}(pp_1), ps \rangle)$ 
18                      $pp_1 \leftarrow \text{next}(pp_1)$ 
19                  $p_1 \leftarrow \text{next}(p_1)$ 
20                  $p_2 \leftarrow \text{next}(p_2)$ 
21             else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
22                 then  $p_1 \leftarrow \text{next}(p_1)$ 
23             else  $p_2 \leftarrow \text{next}(p_2)$ 
24 return answer
```

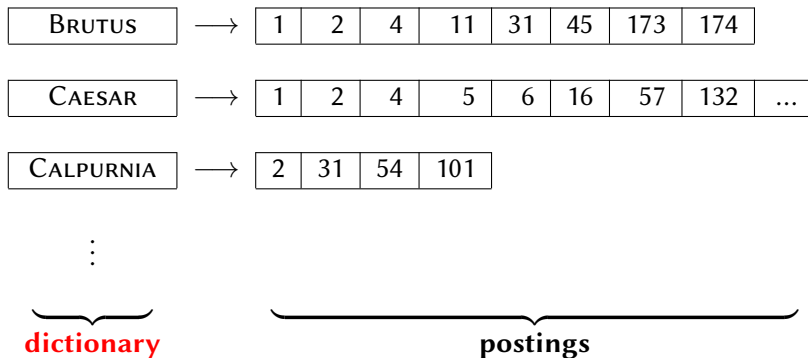
## “Positional” queries on Google

- ▶ For web search engines, positional queries are much more expensive than regular Boolean queries.
- ▶ Let's look at the example of phrase queries.
- ▶ Why are they more expensive than regular Boolean queries?
- ▶ Can you demonstrate on Google that phrase queries are more expensive than Boolean queries?

## Dictionary data structures

## Inverted index

For each term  $t$ , we store a list of all documents that contain  $t$ .



The dictionary is the data structure for storing the term vocabulary.

## Dictionary as array of fixed-width entries

- ▶ For each term, we need to store a couple of items:
  - ▶ document frequency
  - ▶ pointer to postings list
  - ▶ ...
- ▶ Assume for the time being that we can store this information in a fixed-length entry.
- ▶ Assume that we store these entries in an array.

## Dictionary as array of fixed-width entries

Dictionary:

| term   | document<br>frequency | pointer to<br>postings list |
|--------|-----------------------|-----------------------------|
| a      | 656,265               | →                           |
| aachen | 65                    | →                           |
| ...    | ...                   | ...                         |
| zulu   | 221                   | →                           |

Space needed:    20 bytes            4 bytes            4 bytes

1. How do we look up a query term  $q_i$  in this array at query time?
2. Which data structure do we use to locate the entry (row) in the array where  $q_i$  is stored?

## Data structures for looking up term

- ▶ Two main classes of data structures: [hashes](#) and [trees](#).
- ▶ Some IR systems use hashes, some use trees.
- ▶ Criteria for when to use hashes vs. trees:
  1. Is there a fixed number of terms or will it keep growing?
  2. What are the frequencies with which various keys will be accessed?
  3. How many terms are we likely to have?

# Hashes

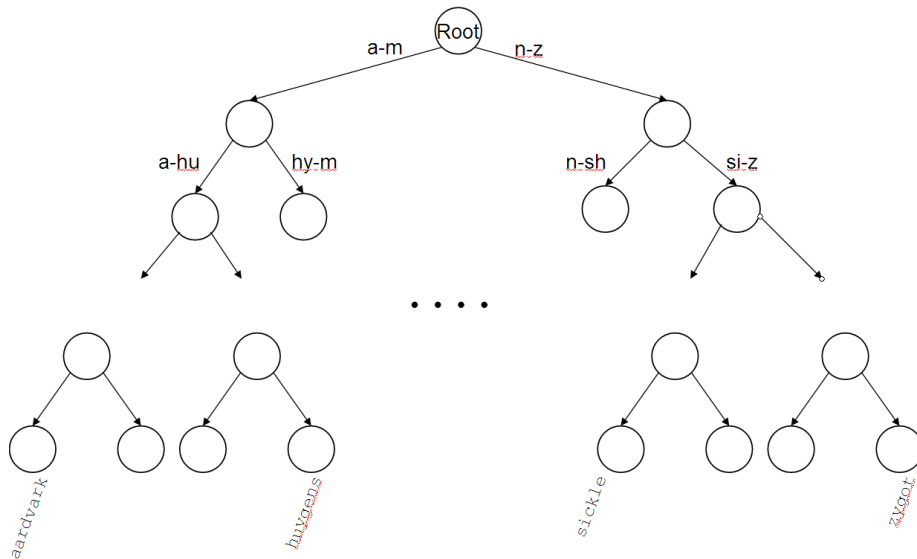
- ▶ Each vocabulary term is hashed into an integer.
- ▶ Try to avoid collisions
- ▶ At query time, do the following: hash query term, resolve collisions, locate entry in fixed-width array
- ▶ Pros:
  1. Lookup in a hash is faster than lookup in a tree.
  2. Lookup time is constant.
- ▶ Cons:
  1. no way to find minor variants (*resume* vs. *résumé*)
  2. no prefix search (all terms starting with *automat*)
  3. need to rehash everything periodically if vocabulary keeps growing



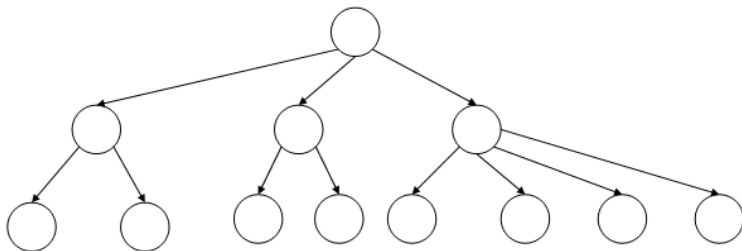
# Trees

- ▶ Trees solve the prefix problem (e.g. find all terms starting with *auto*).
- ▶ Search is slightly slower than in hashes:  $O(\log M)$ , where  $M$  is the size of the vocabulary
- ▶  $O(\log M)$  only holds for **balanced** trees. Rebalancing is expensive.
- ▶ **B-trees** mitigate the rebalancing problem.
- ▶ B-tree definition: every internal node has a number of children in the interval  $[a, b]$  where  $a, b$  are appropriate positive integers, e.g.,  $[2, 4]$ .
- ▶ Simplest tree: binary tree

# Binary tree example



## B-tree example



Tolerant retrieval

## Wildcard queries

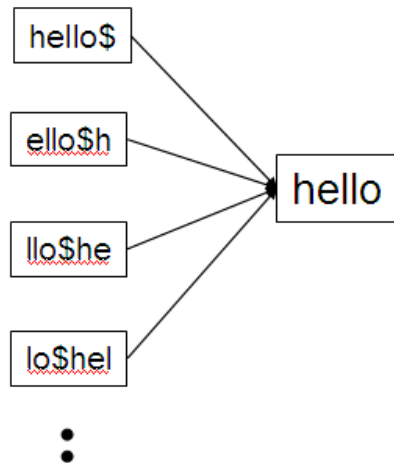
- ▶  $mon^*$ : find all docs containing any term beginning with  $mon$
- ▶ With B-tree dictionary: find all terms  $t$  in the range  $mon \leq t < moo$
- ▶  $*mon$ : find all docs containing any term ending with  $mon$ 
  1. Maintain an additional tree for terms *backwards*
  2. Retrieve all terms  $t$  in the range:  $nom \leq t < non$
- ▶ Result: A set of terms that are matches for wildcard query
- ▶ Then retrieve documents that contain **any of these terms**

## How to handle \* in the middle of a term

- ▶ **Example:**  $m^*nchen$
- ▶ Simple approach: We look up  $m^*$  and  $^*nchen$  in the backward B-tree and intersect the two sets of terms (expensive).
- ▶ Alternative: **permuterm index**

## Permuterm → term mapping

- ▶ Basic idea: Rotate every wildcard query so that \* occurs at the end.
- ▶ Store each of these rotations in the dictionary, say, in a B-tree
- ▶ For term HELLO: add *hello\$*, *ello\$h*, *llo\$he*, *lo\$hel*, and *o\$hell* to the B-tree where \$ is a special symbol



## Permuterm index

- ▶ For HELLO, we've stored: *hello\$, ello\$h, llo\$he, lo\$hel, and o\$hell*
- ▶ Queries:
  - ▶ For X, look up X\$
  - ▶ For X\*, look up \$X\*
  - ▶ For \*X, look up X\$\*
  - ▶ For \*X\*, look up X\*
  - ▶ For X\*Y, look up Y\$X\*
- ▶ Example: For *hel\*o*, look up *o\$hel\**



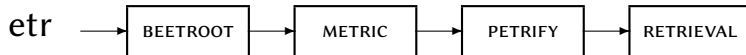
## Processing a lookup in the permuterm index

- ▶ Rotate query wildcard to the right
- ▶ Use B-tree lookup as before
- ▶ Problem: Permuterm more than **quadruples** the size of the dictionary compared to a regular B-tree (empirical estimation).

## $k$ -gram indexes

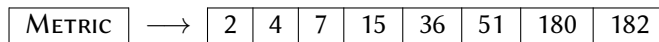
- ▶ More space-efficient than permuterm index
- ▶ Enumerate all character  $k$ -grams (sequence of  $k$  characters) occurring in a term (2-grams are called **bigrams**).
- ▶ Example: from “*April is the cruelest month*” we get the bigrams:  
*\$a ap pr ri il l\$ \$i is s\$ \$t th he e\$ \$c cr ru ue el le es st t\$ \$m mo on nt h\$*
- ▶ \$ is a special word boundary symbol, as before.
- ▶ Maintain an inverted index **from bigrams to the terms** that contain the bigram.

## Postings list in a 3-gram inverted index

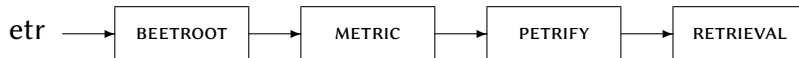


## $k$ -gram (bigram, trigram, ...) indexes

- ▶ Note that we now have two different types of inverted indexes
- ▶ The **term-document inverted index** for finding documents based on a query consisting of terms



- ▶ The  **$k$ -gram index** for finding terms based on a query  $k$ -grams



## Processing wildcarded terms in a bigram index

- ▶ Query *mon*\* can now be run as: *\$m* AND *mo* AND *on*
- ▶ Gets us all terms with the prefix *mon* ...  
...but also many “false positives” like *MOON*.
- ▶ We must postfilter these terms against query.
- ▶ Surviving terms are then looked up in term-document inverted index.
- ▶ *k*-gram index vs. *permuterm* index
  - ▶ *k*-gram index is more space efficient.
  - ▶ Permuterm index doesn't require postfiltering.

## Exercise

- ▶ Google has very limited support for wildcard queries.
- ▶ Query example which doesn't work well on Google: *[gen\* universit\*]*
  - ▶ Intention: you are looking for the University of Geneva, but don't know which accents to use for the French words for university and Geneva.
- ▶ Google treats the asterisk (\*) as a placeholder for 1 or more words
- ▶ But this is not entirely true. Try e.g. *[“pythag\*”]*
- ▶ Exercise: Why doesn't Google fully support wildcard queries?

## Processing wildcard queries in the term-document index

- ▶ **Problem 1:** Potential execution of a large number of Boolean queries.
  - ▶ Most straightforward semantics: Conjunction of disjunctions
  - ▶ For  $[gen^* universit^*]$ : *geneva university* OR *geneva université* OR *genève university* OR *genève université* OR *general universities* OR ...
  - ▶ Very expensive
- ▶ **Problem 2:** Users hate to type.
  - ▶ If abbreviated queries like  $[pyth^* theo^*]$  for *[pythagoras' theorem]* are allowed, users will use them a lot.
  - ▶ This would significantly increase the cost of answering queries.
  - ▶ Somewhat alleviated by Google Suggest

## Positional index

- Phrase queries

- Proximity search

## Dictionary data structures

- Arrays

- Hashes and trees

## Tolerant retrieval

- Wildcard queries

- Permuterm index

- $k$ -gram index